

Ciclo di Vita di un Pod

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Descrivere le cinque fasi del ciclo di vita di un Pod (**Pending**, **Running**, **Succeeded**, **Failed**, **Unknown**) e le relative transizioni
- Interpretare le **PodConditions** per capire lo stato di scheduling e readiness
- Scegliere la corretta **restartPolicy** in base al tipo di workload
- Configurare **livenessProbe**, **readinessProbe** e **startupProbe** per rendere i Pod auto-guarenti
- Diagnosticare e risolvere stati anomali comuni (**CrashLoopBackOff**, **ImagePullBackOff**, **OOMKilled**)
- Interagire con Pod in esecuzione tramite **oc exec**, **oc cp** e **oc port-forward**

Teoria e Concetti Chiave

Cos'è un Pod

Un **Pod** è l'unità di deploy più piccola in Kubernetes e OpenShift. Un Pod racchiude uno o più container che condividono:

- La stessa interfaccia di rete (stessa coppia IP:porta)
- Lo stesso insieme di volumi montati
- Lo stesso namespace di processo (opzionale, tramite **shareProcessNamespace**)

 **Suggerimento:** In produzione la quasi totalità dei Pod contiene un singolo container applicativo. I Pod multi-container usano pattern specifici come init container e sidecar (trattati nel file **05-init-e-sidecar-containers.md**).

Fasi del Ciclo di Vita (Phase)

Il campo **status.phase** di un Pod può assumere i seguenti valori:

Fase	Significato
Pending	Il Pod è stato accettato dal cluster, ma uno o più container non sono ancora in esecuzione (immagine in download, nodo non ancora assegnato, risorse insufficienti)
Running	Il nodo è stato assegnato, tutti i container sono stati creati e almeno uno è ancora in esecuzione, in fase di avvio o in fase di riavvio
Succeeded	Tutti i container del Pod sono terminati con exit code 0 e non verranno riavviati
Failed	Tutti i container sono terminati e almeno uno è terminato con exit code $\neq 0$ o è stato terminato dal sistema
Unknown	Lo stato del Pod non può essere determinato (tipicamente: perdita di comunicazione con il nodo)

Condizioni del Pod (PodConditions)

Le condizioni forniscono una vista più granulare dello stato. Si trovano in `status.conditions`:

Condizione	Descrizione
<code>PodScheduled</code>	Il Pod è stato assegnato a un nodo; <code>False</code> = scheduling bloccato (risorse, taint, affinity)
<code>Initialized</code>	Tutti gli init container sono completati con successo
<code>ContainersReady</code>	Tutti i container nel Pod sono pronti (hanno superato la readinessProbe)
<code>Ready</code>	Il Pod è pronto a servire traffico e può essere aggiunto agli endpoint di un Service

restartPolicy

La `restartPolicy` si applica a tutti i container del Pod e controlla il comportamento in caso di terminazione del container:

Valore	Comportamento	Caso d'uso tipico
<code>Always</code>	Riavvia sempre il container (anche se termina con exit code 0)	Applicazioni web, microservizi — workload continui
<code>OnFailure</code>	Riavvia solo se l'exit code è $\neq 0$	Job batch che devono completarsi con successo
<code>Never</code>	Non riavvia mai	Debug, task one-shot, Job con gestione degli errori esterna

⚠ Attenzione: I `Deployment` supportano solo `restartPolicy: Always`. Per `OnFailure` e `Never` si usano `Job` e `CronJob`.

Probe di Salute

OpenShift supporta tre tipi di probe configurabili per ogni container:

Probe	Scopo	Azione in caso di fallimento
<code>livenessProbe</code>	Verifica che il container sia ancora vivo	Kubernetes riavvia il container
<code>readinessProbe</code>	Verifica che il container sia pronto a ricevere traffico	Il Pod viene rimosso dagli endpoint del Service
<code>startupProbe</code>	Dà tempo al container di avviarsi prima che scattino le altre probe	Disabilita <code>liveness</code> e <code>readiness</code> durante l'avvio lento

Ogni probe supporta tre meccanismi di verifica:

```
# Verifica HTTP (status code 200-399)
httpGet:
  path: /healthz
```

```

port: 8080
scheme: HTTP

# Verifica TCP (porta aperta)
tcpSocket:
  port: 5432

# Esecuzione comando interno al container
exec:
  command: ["cat", "/tmp/healthy"]

```

Parametri comuni delle probe:

Parametro	Default	Descrizione
<code>initialDelaySeconds</code>	0	Secondi di attesa prima del primo controllo
<code>periodSeconds</code>	10	Frequenza dei controlli
<code>timeoutSeconds</code>	1	Timeout per ogni singolo controllo
<code>failureThreshold</code>	3	N° di fallimenti consecutivi prima dell'azione
<code>successThreshold</code>	1	N° di successi consecutivi per tornare healthy

terminationGracePeriodSeconds e Shutdown Graceful

Quando un Pod viene eliminato, Kubernetes invia `SIGTERM` al processo principale del container. Il container ha `terminationGracePeriodSeconds` (default: 30) per terminare in modo pulito. Dopo questo intervallo viene inviato `SIGKILL`.

```

spec:
  terminationGracePeriodSeconds: 60 # aumentato per applicazioni che necessitano
  di più tempo
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal

```

Stati Anomali Comuni

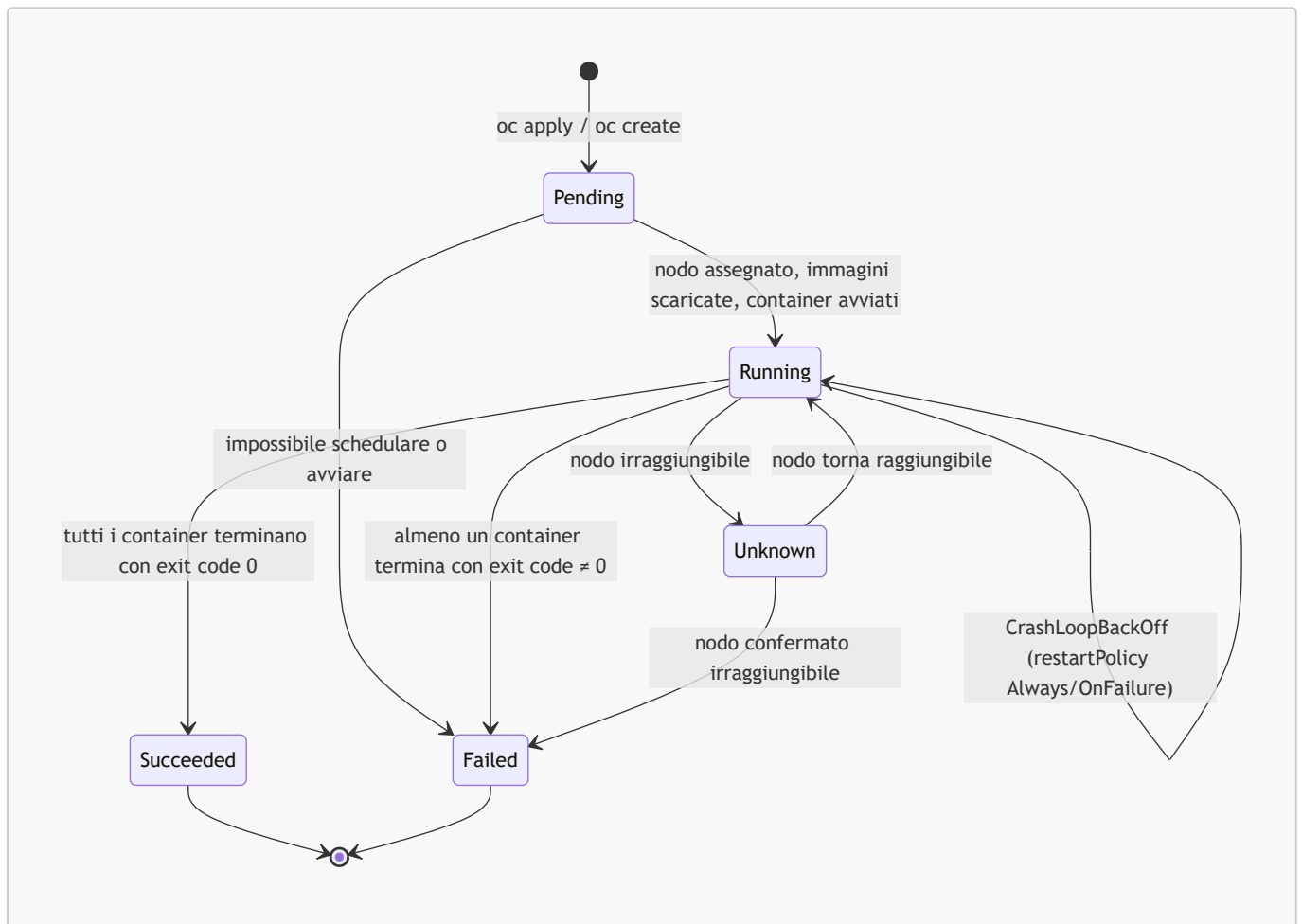
Stato	Causa probabile	Come diagnosticare
<code>CrashLoopBackOff</code>	Il container si avvia e poi crasha ripetutamente	<code>oc logs <pod> --previous, oc describe pod <pod></code>
<code>ImagePullBackOff</code>	Impossibile scaricare l'immagine container	Credenziali mancanti, nome immagine errato, registry irraggiungibile
<code>OOMKilled</code>	Il container ha superato il limite di memoria	<code>oc describe pod</code> → Events, aumentare <code>resources.limits.memory</code>

Stato	Causa probabile	Come diagnosticare
Pending permanente	Nessun nodo soddisfa i requisiti di scheduling	<code>oc describe pod</code> → Events: Insufficient cpu/memory, taint, affinity
Terminating permanente	Finalizer non rimosso o nodo irraggiungibile	<code>oc delete pod <pod> --grace-period=0 -force</code> (con cautela)

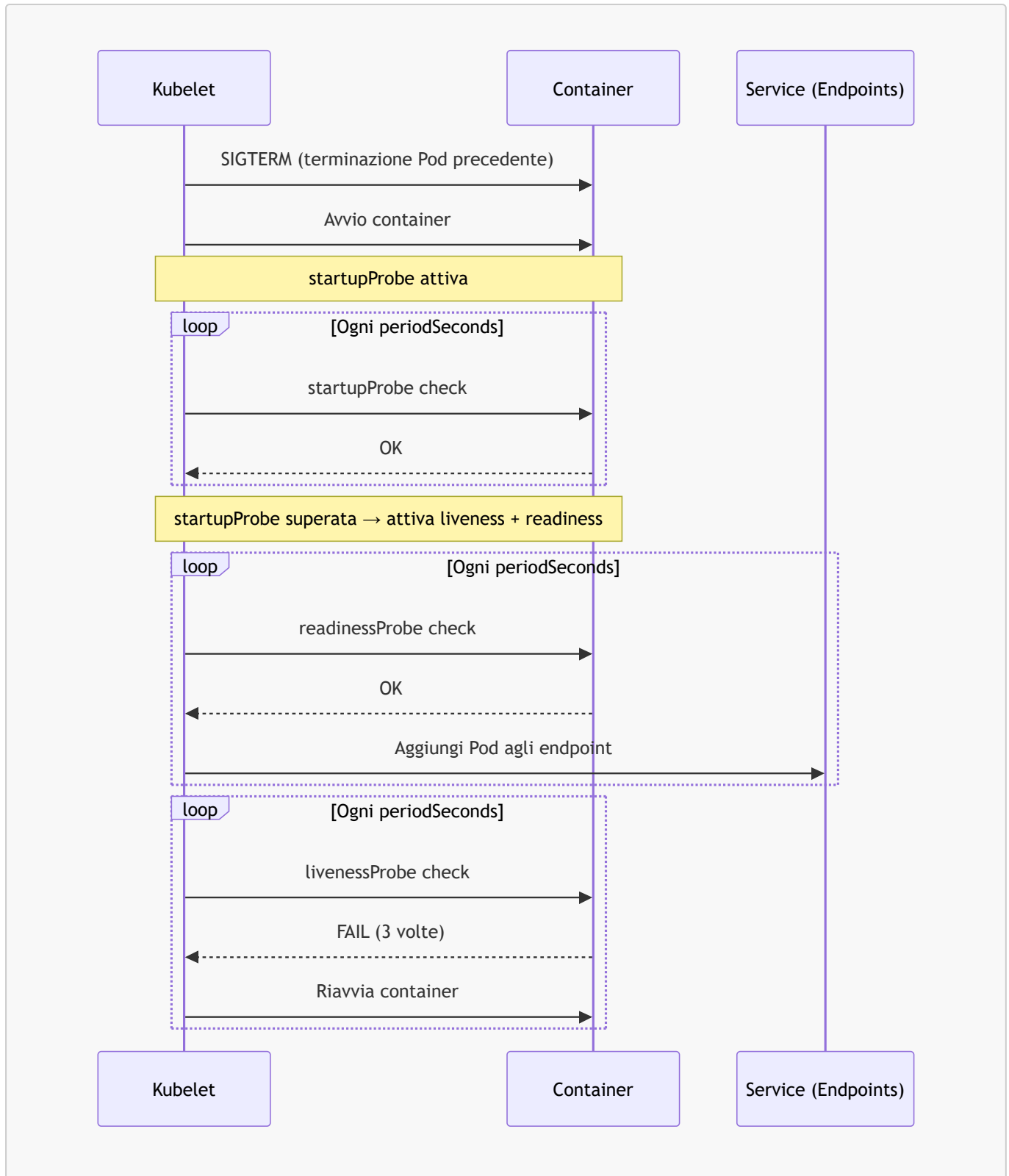
Diagrammi

Ciclo di Vita del Pod — State Diagram

Il seguente diagramma mostra le transizioni di fase di un Pod dall'invio al cluster alla sua terminazione:



Flusso delle Probe



Comandi Pratici con Output Atteso

```
# Elencare tutti i Pod nel progetto corrente con stato esteso
oc get pods -o wide
```

```
# Output atteso:
```

```
# NAME                                READY   STATUS    RESTARTS   AGE   IP
```

```
NODE
```

```
# webapp-6d9f7b8c9-xk2lp      1/1      Running    0          12m    10.128.2.45
worker-0
# backend-db-0                1/1      Running    0          45m    10.128.2.47
worker-1
```

Mostrare i dettagli completi di un Pod inclusi eventi e condizioni
oc describe pod webapp-6d9f7b8c9-xk2lp

Output atteso (estratto rilevante):

Status: Running

Conditions:

Type Status

Initialized True

Ready True

ContainersReady True

PodScheduled True

Events:

Normal Scheduled 13m scheduler Successfully assigned
myproject/webapp-... to worker-0

Normal Pulling 13m kubelet Pulling image
"registry.access.redhat.com/ubi9/ubi-minimal"

Normal Started 12m kubelet Started container webapp

Seguire i log di un Pod in tempo reale

oc logs -f webapp-6d9f7b8c9-xk2lp

Leggere i log del container precedente (utile dopo CrashLoopBackOff)

oc logs webapp-6d9f7b8c9-xk2lp --previous

Aprire una shell interattiva nel container del Pod

oc exec -it webapp-6d9f7b8c9-xk2lp -- /bin/bash

Output atteso:

bash-5.1\$

Eseguire un singolo comando nel container senza shell interattiva

oc exec webapp-6d9f7b8c9-xk2lp -- env | grep APP_

Output atteso:

APP_ENV=production

APP_PORT=8080

```
# Copiare un file dal container alla macchina locale
oc cp webapp-6d9f7b8c9-xk2lp:/app/logs/app.log ./app.log

# Copiare un file dalla macchina locale al container
oc cp ./config.yaml webapp-6d9f7b8c9-xk2lp:/app/config/config.yaml
```

```
# Fare port-forward per accedere al Pod da localhost (utile per debug)
oc port-forward webapp-6d9f7b8c9-xk2lp 8080:8080

# Output atteso:
# Forwarding from 127.0.0.1:8080 -> 8080
# Forwarding from [::1]:8080 -> 8080
```

```
# Visualizzare lo YAML completo di un Pod in esecuzione
oc get pod webapp-6d9f7b8c9-xk2lp -o yaml
```

```
# Monitorare i Pod in tempo reale (aggiornamento continuo)
oc get pods --watch

# Output atteso:
```

# NAME	READY	STATUS	RESTARTS	AGE
# webapp-6d9f7b8c9-xk2lp	0/1	Pending	0	2s
# webapp-6d9f7b8c9-xk2lp	0/1	Init:0/1	0	3s
# webapp-6d9f7b8c9-xk2lp	0/1	Running	0	8s
# webapp-6d9f7b8c9-xk2lp	1/1	Running	0	15s

```
# Eliminare un Pod (verrà ricreato dal controller se gestito da un Deployment)
oc delete pod webapp-6d9f7b8c9-xk2lp

# Forzare l'eliminazione di un Pod bloccato in Terminating (usare con cautela)
oc delete pod webapp-6d9f7b8c9-xk2lp --grace-period=0 --force
```

Esempi YAML Completi

Pod con Probe e restartPolicy

```
apiVersion: v1
kind: Pod
metadata:
  name: webapp-demo
```

```
namespace: myproject
labels:
  app: webapp
  tier: frontend
spec:
  # Politica di riavvio: Always per servizi continui
  restartPolicy: Always
  # Tempo concesso per terminare in modo pulito prima del SIGKILL
  terminationGracePeriodSeconds: 30
  containers:
    - name: webapp
      image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
      ports:
        - containerPort: 8080
          protocol: TCP
      # Limiti di risorse obbligatori su ARO per evitare OOMKilled
      resources:
        requests:
          memory: "64Mi"
          cpu: "50m"
        limits:
          memory: "128Mi"
          cpu: "200m"
      # startupProbe: dà al container fino a 5*30=150s per avviarsi
      startupProbe:
        httpGet:
          path: /healthz
          port: 8080
        failureThreshold: 5
        periodSeconds: 30
        initialDelaySeconds: 10
      # livenessProbe: riavvia il container se smette di rispondere
      livenessProbe:
        httpGet:
          path: /healthz
          port: 8080
        initialDelaySeconds: 0
        periodSeconds: 10
        timeoutSeconds: 2
        failureThreshold: 3
      # readinessProbe: rimuove il Pod dal Service se non pronto
      readinessProbe:
        httpGet:
          path: /ready
          port: 8080
        initialDelaySeconds: 5
        periodSeconds: 5
        failureThreshold: 3
      env:
        - name: APP_ENV
          value: "production"
```


Pod con livenessProbe TCP e exec

```
apiVersion: v1
kind: Pod
metadata:
  name: db-probe-demo
  namespace: myproject
spec:
  restartPolicy: Always
  containers:
  - name: postgres
    image: registry.redhat.io/rhel9/postgresql-15:latest
    ports:
    - containerPort: 5432
    resources:
      requests:
        memory: "256Mi"
        cpu: "100m"
      limits:
        memory: "512Mi"
        cpu: "500m"
    # livenessProbe via TCP: verifica che la porta 5432 sia aperta
    livenessProbe:
      tcpSocket:
        port: 5432
      initialDelaySeconds: 30
      periodSeconds: 10
    # readinessProbe via exec: esegue pg_isready nel container
    readinessProbe:
      exec:
        command:
        - pg_isready
        - -U
        - postgres
      initialDelaySeconds: 10
      periodSeconds: 5
```

Note Specifiche per Azure ARO

SCC e Pod Security

Su ARO, i Pod vengono eseguiti con SCC (Security Context Constraints) restrittive per impostazione predefinita. Il SCC `restricted-v2` (default su OCP 4.14+) non permette:

- Container che girano come root
- Uso di `hostNetwork`, `hostPID`, `hostIPC`
- Mounting di path arbitrari dell'host

Per verificare quale SCC viene applicata a un Pod:

```
oc get pod webapp-demo -o yaml | grep -i scc
# oppure
oc describe pod webapp-demo | grep scc
```

⚠ **Attenzione:** Se un Pod su ARO rimane bloccato in **Pending** con errori di tipo **unable to validate against any security context constraint**, verificare il SCC del ServiceAccount associato con **oc adm policy who-can use scc restricted-v2**.

Resource Limits su ARO

Su ARO è consigliato (e spesso obbligatorio nei namespace con LimitRange applicato) specificare sempre **resources.requests** e **resources.limits**:

```
# Verificare se esiste un LimitRange nel namespace
oc get limitrange -n myproject

# Output atteso:
# NAME                      CREATED AT
# core-resource-limits      2024-01-15T10:00:00Z
```

Probe e Health Check con Azure Load Balancer

Quando un Pod è esposto tramite un Service di tipo **LoadBalancer** su ARO (backed da Azure Load Balancer), la **readinessProbe** è critica: un Pod non ready viene automaticamente rimosso dal backend pool di Azure. Configurare sempre **readinessProbe** per evitare traffico verso container non pronti.

Riepilogo dei Punti Chiave

- Un Pod transita tra le fasi **Pending** → **Running** → **Succeeded/Failed**; la fase **Unknown** indica perdita di contatto con il nodo
- Le **PodConditions** (**PodScheduled**, **Initialized**, **ContainersReady**, **Ready**) forniscono dettagli granulari sullo stato del Pod
- **restartPolicy**: **Always** per servizi continui, **OnFailure** per Job, **Never** per task one-shot
- **livenessProbe** riavvia il container se non risponde; **readinessProbe** lo rimuove dal Service; **startupProbe** protegge l'avvio lento
- **CrashLoopBackOff** si diagnostica con **oc logs --previous**; **OOMKilled** richiede aumento dei **limits.memory**
- **terminationGracePeriodSeconds** determina quanto tempo il container ha per chiudersi in modo pulito dopo **SIGTERM**
- Su ARO, verificare sempre SCC e LimitRange prima di deployare Pod con requisiti di sicurezza non standard

Domande di Verifica

1. Un Pod è in fase `Running` ma il Service non instrada traffico verso di esso. Qual è la causa più probabile?

- A) La `livenessProbe` sta fallendo
- B) La `readinessProbe` sta fallendo ✓
- C) Il Pod ha `restartPolicy: Never`
- D) Il Pod ha superato il `terminationGracePeriodSeconds`

2. Quale comando permette di leggere i log del container che è crashato prima del riavvio corrente?

- A) `oc logs <pod> --last`
- B) `oc logs <pod> --restart`
- C) `oc logs <pod> --previous` ✓
- D) `oc describe pod <pod>`

3. Un container viene continuamente riavviato con stato `OOMKilled`. Qual è la soluzione corretta?

- A) Aumentare `resources.requests.cpu`
- B) Aumentare `resources.limits.memory` ✓
- C) Impostare `restartPolicy: Never`
- D) Aggiungere una `startupProbe`

4. Qual è la differenza tra `livenessProbe` e `readinessProbe`?

- A) Non c'è differenza: entrambe riavviano il container
- B) `livenessProbe` riavvia il container; `readinessProbe` rimuove il Pod dagli endpoint del Service ✓
- C) `readinessProbe` riavvia il container; `livenessProbe` rimuove il Pod dagli endpoint del Service
- D) `livenessProbe` funziona solo con HTTP; `readinessProbe` solo con TCP

5. Un Pod rimane in fase `Pending` con l'evento "`Insufficient memory`". Cosa è necessario fare?

- A) Aumentare `resources.limits.memory` del Pod
- B) Ridurre `resources.requests.memory` del Pod oppure aggiungere nodi con più memoria disponibile ✓
- C) Aggiungere una `livenessProbe`
- D) Impostare `restartPolicy: OnFailure`